



***Geidea – KSA WEB ECR
V1.09***

Version	Date	Author	Changes
1.08	27-01-2026	Mohamed Shalik	Batch File Installation of a service.
1.09	10-02-2026	Mohamed Shalik	Responsibility Column Added

Table of Contents:

1	Introduction	3
2	Architecture Diagram.....	4
3	Prerequisites & Merchant Responsibility	4
4	Integration flow	5
4.1	Flow Diagram	5
4.2	Integration Flow Overview.....	6
5	Geidea Terminal Integration: Implementation & Protocol Overview	7
6	Technical Implementation	9
6.1	Connection with Window Service.....	9
6.2	Connect with Geidea Terminal Over Websocket	10
6.3	Websocket Response JSON Format	18
7	Installation and Trouble Shooting Guide.....	22
8	Terminal and SAMA Response Messages (Socket level).....	25
8.1	Terminal Response Messages	25
8.2	SAMA Response Messages	29

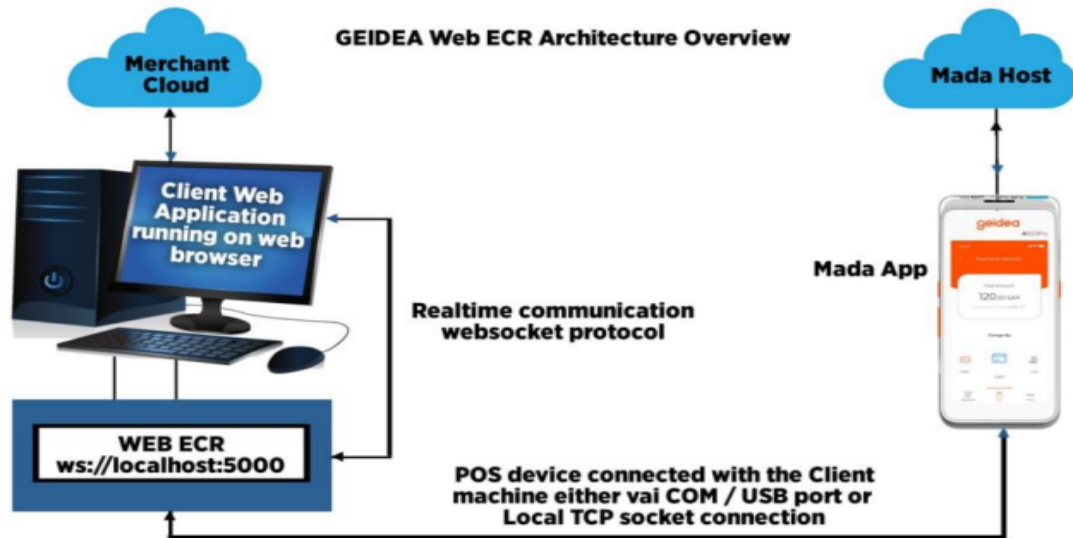
1. Introduction

- This documentation will show how Geidea terminal can be integrated with client web-based ECR.
- For testing purposes, we've developed **GEIDEA_WEBECR** testing Tool that can be utilized for the purpose of establishing communication with Geidea POS. It functions as a web-based application, specifically designed for the Windows operating system.
- "Geidea_WebECR" application can be generated by utilizing the binary files provided by Geidea.
- The primary function of this service is to facilitate communication between the ECR (a web-based application running on a local personal computer) and Geidea POS.

Note:

It is important to note that the use of Geidea_WebECR involves a one-to-one communication.

2. Architecture Diagram

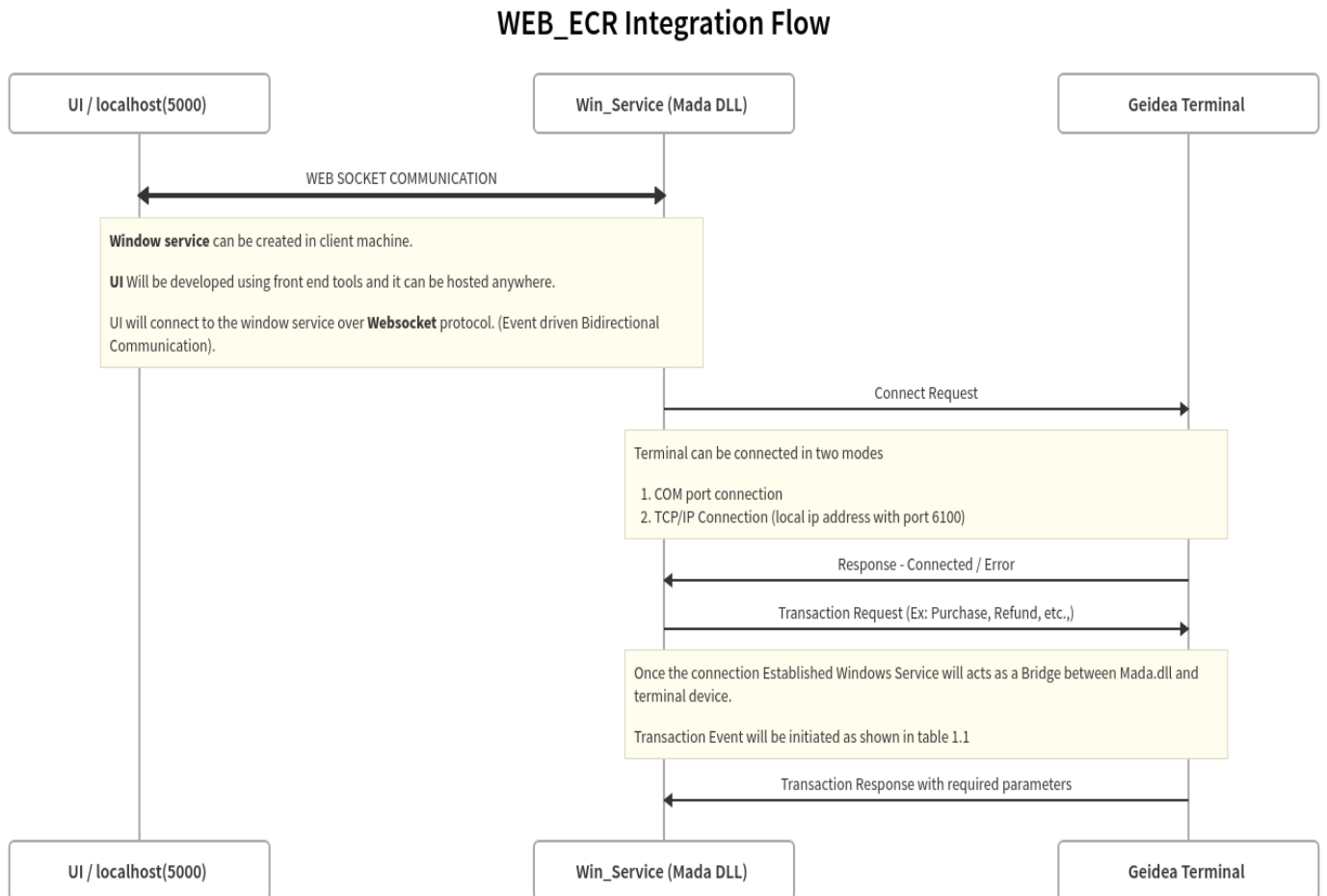


3. Prerequisites & Merchant Responsibility

- Windows 7 and above.
- Local host with 5000,5001 port access.
- Merchant Responsibilities :
 - Since the Geidea Web ECR operates as a one-to-one solution, the Geidea Web ECR Service must be installed on each client cashier PC (ECR). The installation of this service is the responsibility of the merchant. As outlined in Section 7, the merchant is authorized to install and configure the service accordingly.

4. Integration Flow

4.1 Flow Diagram



MADE WITH swimlanes.io

4.2 Integration Flow Overview:

We have successfully developed a POC using dotnet core for web ECR integration. In order to seamlessly integrate the solution, the below steps need to be followed:

- First:** install our solution on each client PC as a Windows service.
- Second:** ensure that the port 5000 is open for inbound traffic.
The Windows service will be listening on this port at the localhost of that particular computer.
- Third:** connect our POS device to the PC via a COM port once the installation process is complete. (If the connection is TCP, a wireless network can be used, but only on a local network basis. The internet will not work).

All transaction requests to the POS device must be sent through the client Windows service and the communication with the Windows service I/O operations must be done using the WebSocket protocol.

The flow of operation will be from your software to the Windows service over websocket to the POS machine through TCP/COM port and back to the Windows service over websocket to your software.

All request and response data will be in JSON format from your software to the Windows service over web socket protocol.

Note:

this solution will only work on a local network and not over the internet.

5. Geidea Terminal Integration: Implementation & Protocol Overview:

- The application was created utilizing dotnet core version 6 and relies on the WebSocket Protocol for the purpose of communication.
- The software will be implemented on every individual client device as a Windows service and executed locally on port 5000.
- Client applications are expected to employ a websocket protocol to establish a connection and engage in communication with the Windows service.
- The connection between client PCs and Geidea terminal device should be established through the com port. In the case of **TCP/IP** connection, it is possible to connect to the terminal over the LAN IP assigned to that POS device. In this scenario, a COM Port connection is not required.
- The client application should initiate the connection with the Windows service. Once connected, the Windows service will commence emitting the events to the client application using websocket protocol. The client application should attentively listen and respond accordingly.

The working cycle will proceed as follows:

- The client applications will establish a connection with the Windows service. Once connected, it will send a request to retrieve a list of available COM ports that can be connected.
- At this point, the client application is aware of the list of Com ports that are connected or ready to connect to the terminal devices and needs to send requests to the windows service to initiate a connection with the selected Geidea terminal. This is done by providing inputs such as the **comName**, **baudRate**, **dataBits**, **stopBits**, and **parity**.
- If the connection is TCP/IP, the LAN IP of the POS terminal and the ECR port on the POS device should be provided as input for the **IpAddress** and **Port** respectively.
- Each request to the Windows service is sent using the **socket.send()** method, and every event from the Windows service is listened to on the socket onmessage event in the client application. The window service distinguishes events by sending different data payloads to the client application in the socket **onmessage** event.
- Once the Windows service establishes a connection with Geidea terminal device, it will emit **onConnector** and **onError** events to the client, responding to the terminal connection request in the form of JSON data payload.
- The JSON payload will contain the necessary response values for transaction requests/responses, terminal actions, terminal status, and HTML receipts.

It is recommended to use the pub/sub pattern in the client application to listen and respond to events accordingly, with geideaGUI serving as a reference.

Note:

The port may vary depending on the device model, so it is advisable to consult with a POS technician to determine the ECR port for TCP connection.

For TCP/IP connection it is recommended to use to use connect operation as a distinct operation.

6. Technical Implementation

6.1 Connection with Windows Service

- To establish a connection with the window service, the client application must possess a WebSocket client. Additionally, it is necessary for the Geidea window service to be installed and operational on the local machine.
- In the event that the Geidea windows service is running on the localhost port 5000, the client must establish a connection with it by utilizing the following Javascript code:

```
const socket = new WebSocket(`ws://localhost:5000/messages`);
```

- Upon successful connection, the window service will acknowledge the client application by triggering the onconnect event.

```
socket.onopen = function ()
{
  EventEmitter.dispatch("PORT_OPEN", "port opened :");
};
```

- The client application, as mentioned earlier, is required to actively monitor and respond to the **onclose** and **onmessage** events. This is necessary in order to receive notifications regarding the termination of the connection and the arrival of new messages from the associated Windows service

```
socket.onmessage = function (data)
{
  console.log({ data }, "onmessage");
  EventEmitter.dispatch("PORT_DATA", data);
};
socket.onclose = function (data)
{
  EventEmitter.dispatch("CONNECTION_CLOSED", data);
};
```

Note:

the client application will be provided with the data in the form of a stringified JSON response. Furthermore, it is essential to serialize the data in order to obtain the JSON data for further processing.

6.2 Connect with Geidea Terminal over WebSocket

- We have split the events into two types for this integration **CONNECTION** and **TRANSACTION** respectively. As the name describes, if the events are related to connection with Windows service/geidea terminal are considered as CONNECTION events. If the events are related to transaction or terminal operations, it will be considered as TRANSACTION events.
- To connect with the terminal over websocket, client application needs to send the request with the following payload. Below is the example payload for connecting to POS Terminal using COM port.

```
socket.send( JSON.stringify(
  {
    Event: "CONNECTION",
    Operation: "CONNECT",
    connectionMode,
    comName,
    baudRate,
    dataBits,
    parity
  })
);
```

- Here Operation is the 2nd subtype used to differentiate the request type to the Windows service. For example, to connect, it will be **CONNECT**, to disconnect, it will be **DISCONNECT**, to initiate purchase operation, it will be **PURCHASE**, to process the refund operation, it will be **REFUND**. connectionMode should be either COM or TCP. Below is an example of a payload for connecting to the POS Terminal using TCP/IP.

```
socket.send( JSON.stringify(
  {
    Event: "CONNECTION",
    Operation: "CONNECT",
    connectionMode: "TCP",
    ipAddress,
    port
  })
);
```

Note:

you have to stringify the JSON payload by `JSON.stringify(JSONData)` before sending it to the socket. Socket will accept string type only. Same you have to do for response handling in `onmessage` as well. You will receive the data as stringified JSON. you further need to parse it to JSON by `JSON.parse(responseData)`.

Important Notes:

- Socket will accept string type only. Same should be applied for response handling in onmessage as well.
- Data will be received as stringified JSON. further need to parse it to JSON by `JSON.parse(responseData)`.
- It is highly recommended to establish TCP/IP connections through a button operation, In case where the client wishes to exclusively use TCP/IP as the connecting option, it is imperative to consider the following TCP/IP connection flow.
- There are two main events in the entire flow “CONNECT” and “TRANSACTION” each to be called separately. Ex: To do a Transaction POS device to be connected first to perform any transaction first we need to establish a connection.
- Upon completion of the purchase transaction, the connection will be briefly interrupted and then automatically re-established, hence, there is no need to create a new TCP/IP connection.

WebSocket Events:

Event	Operation	Description	Subsequent Events	Example request JSON Payload
CONNECTION	CONNECT	To connect to geidea terminal device <u>Request Payload</u> connectionMode =COM and comName, braudRate, dataBits, parity are the required params. All are string data types	onConnect Or onError	If COM port connection <pre>{ "Event": "CONNECTION", "Operation": "CONNECT", "ConnectionMode": "COM", "ComName": "COM1", "BraudRate": "38400", "DataBits": "8", "Parity": "none" }</pre> If TCP/IP connection, <pre>{ "Event": "CONNECTION", "Operation": "CONNECT", "IpAddress": "192.168.8.86", "ConnectionMode": "TCP", "Port": 6100 }</pre>
CONNECTION	DISCONNECT	To disconnect from Geidea Terminal device	onDisconnect or onError	<pre>{ "Event": "CONNECTION", "Operation": "DISCONNECT" }</pre>
CONNECTION	GET_TERMINAL_SETTINGS	To get the terminal settings like menu enabled or not, print enabled or not, and baud rate value.		<pre>{ "Event": "CONNECTION", "Operation": "GET_TERMINAL_SETTINGS" }</pre>

TRANSACTION	GET_TERMINAL_ID	To get the terminal id of the connected geidea terminal	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "GET_TERMINAL_ID" }
TRANSACTION	AVAILABLE_TERMINAL_PORTS	To list the com port which are connected with geidea terminal device	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "AVAILABLE_TERMINAL_PORTS" }
TRANSACTION	PURCHASE	To initiate purchase transaction 1. Amount => string value(eg: 10.00 for 10 SAR). It will accept only 2 digits after decimal point. 2. ECRNumber => string (16digit). 3. PrintSettings => string 1 4. AppId => string default 11.	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "PURCHASE", "Amount": "10.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "AppId": "11" }
TRANSACTION	PURCHASE_WITH_SUBSCRIPTION	To initiate a purchase transaction with subscription number. 1. Amount => string value(eg: 10.00 for 10 EGP) 2. ECRNumber => string (16digit). 3. PrintSettings => string 1 4. AppId => string default 11. 5. SubscriptionNumber => string (5	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "PURCHASE_WITH_SUBSCRIPTION", "Amount": "10.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "AppId": "11", "SubscriptionNumber": "A1234" }

		digit alpha-numeric		
TRANSACTION	PURCHASE_WITH_CASHBACK	To initiate a purchase transaction with cashback value <u>Required Payload</u> 1. Amount => string (eg: 10.00 for 10 SR) 2. ECRNumber => string (16digit). 3. PrintSettings => string 1. 4. Appld => string default 11. 5. CashbackAmount => string (eg: 10.00 for 10 SR)	OnDataReceive or onError	<pre>{ "Event": "TRANSACTION", "Operation": "PURCHASE_WITH_CASHBACK", "Amount": "100.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "Appld": "11", "CashbackAmount": "10.00" }</pre>
TRANSACTION	REFUND	To initiate a refund operation <u>Required Payload</u> 1. Amount => string (eg: 00 for 10 SR) 2. ECRNumber => string(16digit). 3. PrintSettings => string 1. 4. Appld => string default 5. RRNNumber => string (12 digits) 6. TransactionDate => string (8 digits)	OnDataReceive or onError	<pre>{ "Event": "TRANSACTION", "Operation": "REFUND", "Amount": "100.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "Appld": "11", "RRNNumber": "123456789012", "TransactionDate": "31122019" }</pre>

		7. (e.g: DDMMYYYY => 8. 31/12/2019)		
TRANSACTION	REFUND_WITH_CARDNUMBER	To initiate a refund operation with card number as a parameter. Required Payload: 1. <u>Amount => string (eg: 00 for 10 SR)</u> 2. <u>ECRNumber => string(16digit).</u> 3. <u>PrintSettings => string 1.</u> 4. <u>Appld => string default</u> 5. <u>RRNNumber => string (12 digits)</u> 6. <u>TransactionDate => string (8 digits)(e.g: DDMMYYYY =>31/12/2019)</u> 7. <u>CardNumber => string</u> 8. <u>(16 digit card number)</u>	OnDataReceive or onError	<pre>{ "Event": "TRANSACTION", "Operation": "REFUND_WITH_CARDNUMBER", "Amount": "100.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "Appld": "11", "RRNNumber": "123456789012", "TransactionDate": "31122019", "CardNumber": "476173*****0010" }</pre>
TRANSACTION	AUTHORIZATION	To initiate authorization transaction using geidea terminal device (AFD) Required Payload 1. <u>Amount => string (eg: 00 for 10 SR)</u> 2. <u>ECRNumber => string (16digit).</u>	OnDataReceive or onError	<pre>{ "Event": "TRANSACTION", "Operation": "AUTHORIZATION", "Amount": "100.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "Appld": "11" }</pre>

		3. <u>PrintSettings => string 1.</u> 4. <u>Appld => string default 11.</u>		
TRANSACTION	ADVISE_TRANSACTION	To initiate advice transaction using geidea terminal device <u>Required Payload:</u> 1. Amount => string (eg: 10.00 for 10 SR) 2. ECRNumber => string (16digit). 3. PrintSettings => string 1. 4. Appld => string default 11.	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "ADVISE_TRANSACTION", "Amount": "100.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "Appld": "11" }
TRANSACTION	REVERSAL	To do a reversal operation Against purchase transaction <u>Required Payload:</u> 1. ECRNumber => string 2. Stan=>string (6digit) 3. Appld => string default 11. 4. Print Settings=>string default 1.	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "REVERSAL", "Appld": "11", "PrintSettings": "1", "ECRNumber": "1234567890123456" }
TRANSACTION	GET_MERCHANT_INFO	To get the merchant information of the connected geidea terminal	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "GET_MERCHANT_INFO" }

TRANSACTION	RECONCILIATION	To initiate reconciliation operation from the geidea terminal device.	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "RECONCILIATION", "PrintSettings": "1" }
TRANSACTION	CHECK_STATUS	To check whether the connected geidea terminal is busy or available to use?	OnTerminalStatus or onError	{ "Event": "TRANSACTION", "Operation": "CHECK_STATUS" }
TRANSACTION	CASH_ADVANCE	To initiate the cash advance transaction using geidea terminal device. <u>Required Payload:</u> 1. Amount => string (eg: 10.00 for 10 SR) 2. ECRNumber => string (16digit). 3. PrintSettings => string 1. 4. Appld => string default 11.	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "CASH_ADVANCE", "Amount": "100.00", "ECRNumber": "1234567890123456", "PrintSettings": "1", "Appld": "11" }
TRANSACTION		To get the previous terminal response data once again	OnDataReceive or onError	{ "Event": "TRANSACTION", "Operation": "REPEAT" }

Each of the above events will be used to send a request to windows service through the websocket protocol and then the windows service will further convert those request payloads into terminal's readable request format and keep listening for the callback events from the terminal device and notify the client application over websocket onmessage event with stringified JSON response.

Each of the request events will have the callback or error event to return the response of that particular request to the geidea terminal through windows service.

6.3 Websocket Response Json Format

- For every request to the windows service `socket.send("<stringified json>")`, it will respond via `socket.onmessage("<stringified json>")` event with stringified json as response data.
- Each of the onmessage events will contain a stringified JSON and each will have an Event field in root level JSON data and it's used to differentiate the callback events as follows.

Event	Description	Sample JSON Response
onConnect	It will be sent to client applications from the windows service once the terminal connection request is successful	<pre>{ "IsConnected": "True", "ConnectionMode": "COM", "Status": "PORT_OPENED", "Message": "COM3", "Event": "OnConnect" }</pre>
onDisconnect	It will be sent to client application if the connection between the geidea terminal device and windows service is closed or lost	<pre>{ "IsConnected": "False", "ConnectionMode": "COM", "Status": "PORT_CLOSED", "Message": "COM3", "Event": "OnDisconnect" }</pre>
onError	It will be sent to client application if any error occurred in communication between the geidea terminal device and windows.	<pre>{ "Message": "Readable error message", "Reason": "root cause for the error occurred", "StackTrace": "stack trace details in the .dll code level", "Event": "OnError" }</pre>
OnTerminalAction	This is to notify the client application about the geidea terminal actions like	<pre>{ "EventName": "TERMINAL_ACTION",</pre>

	SWIPE_OR_INSERT, PIN_ENTRY and PIN_QUIT. These actions are used to describe the client application about the current state of the geidea terminal device. These are event-driven and asynchronous event callbacks from the geidea terminal. Here, EventName is the same for all types of terminal actions, and TerminalAction is the enum of the particular terminal action and the OptionalMessage is the descriptive text message for that particular terminal action. JsonResult will hold all the above mentioned keys and values as stringified JSON format, finally Event, for terminal action events, it will be OnTerminalAction.	<pre>"TerminalAction": "SWIPE_OR_INSERT", "OptionalMessage": "swipe or insert", "JsonResult": "{ \"EventName\": \\\"TERMINAL_ACTION\\\", \"TerminalAction\": \\\"SWIPE_OR_INSERT\\\", \"OptionalMessage\": : \\\"swipe or insert\\\" }", "Event": "OnTerminalAction" }</pre>
onTerminalStatus	This is to notify the client application about whether the geidea terminal is busy or available to accept the transaction request. Basically, this will be sent to the client application after the CHECK_STATUS request or when the client application sends requests to the terminal while it's already busy with an ongoing transaction	A) <u>Busy State Response</u> <pre>{ "EventName": "TERMINAL_STATUS", "TerminalStatus": "BUSY", "OptionalMessage": "Terminal is BUS now", "JsonResult": "{ \\\"EventName\\\": \"TERMINAL_STATUS\\\", \\\"TerminalStatus\\\": \"BUSY\\\", \\\"OptionalMessage\\\": \"Terminal is BUS now\\\" }"; "Event": "OnTerminalStatus" }</pre> B) <u>Available State Response</u> <pre>{ "EventName": "TERMINAL_STATUS", "TerminalStatus": "AVAILABLE", "OptionalMessage": "Terminal is AVAILABLE now", "JsonResult": "{ \\\"EventName\\\": \"TERMINAL_STATUS\\\", \\\"TerminalStatus\\\": \"AVAILABLE\\\", \\\"OptionalMessage\\\": \"Terminal is AVAILABLE now\\\" }"; "Event": "OnTerminalStatus" }</pre>
onDataReceive	This is to respond to the client application regarding transaction request once it is	A) <u>Terminal Settings</u> <pre>{</pre>

	finished. This event will be sent to the client application, whenever the windows service receives data from the terminal regarding transaction operations or terminal settings requests	<pre>"EventName":"TERMINAL_SETTINGS", "JsonResult":{"\IsMenuEnabled\":"0\","IsPrinterEnabled\":"1\","BraudRate\":"38400\"}", "Event": "OnDataReceive" }</pre> - 0 Mean disabled and 1 means enabled for IsMenuenabled - 1 default for IsPrinterEnabled
	This is the very important message/event type that will have the most important response data from the terminal regarding transaction requests	B) <u>Terminal Response</u> ** Please refer the screenshot attached below the table
onWrite	Mostly, it's be used to notify the client application with request and response buffer once it's successfully sent or a new buffer received from the geidea terminal device. It's just to display the raw buffer data to the client application before processing it and it will be used as evidence to find out any issues in data after it's processed. If the data successfully received from the terminal device	A. <u>Data Received</u> <pre>{ "EventName":"DATA_RECEIVED", "JsonResult":{ "\Status\":"DATA_RECEIVED\","Message\":"hexadecimal buffer value\","OptionalMessage\":"new data received from terminal\","Event": "OnWrite" }</pre>
	If the data successfully sent to the terminal device from windows service,	B. <u>Write Success</u> <pre>{ "EventName":"WRITE_SUCCESS", "JsonResult":{"\Status\":"WRITE_SUCCESS\","Message\":"hexadecimal buffer value\","OptionalMessage\":"Data Written to COM port successfully\","Event": "OnWrite" }</pre>

**** Terminal Response for onDataReceive Event**

```
{
  "EventName": "TERMINAL_RESPONSE",
  "JsonResult": "{\r\n  \"TransactionResponseEnglish\": \"APPROVED\",\r\n  \"PrimaryAccountNumber\": \"401613*****7329\",\r\n  \"TransactionAmount\": \"12.00\",\r\n  \"TransactionSTAN\": \"000038\",\r\n  \"TransactionDateTime\": \"230509095632\",\r\n  \"CardExpiryDate\": \"10/25\",\r\n  \"RetrievalReferenceNumber\": \"312909563206\",\r\n  \"TransactionAuthCode\": \"329200\",\r\n  \"TransactionResponse\": \"000\",\r\n  \"CardAcceptorTerminalId\": \"9011100312345678\",\r\n  \"CardAcceptorName\": \"5643|VC-VISA|VISA|فيزا\",\r\n  \"CardAcceptorIdCode\": \"650999000911\",\r\n  \"AdditionalAmount\": \"0.00\",\r\n  \"BankId\": \"SABB\",\r\n  \"CardAcceptorBusinessCode\": \"5651\",\r\n  \"DateTime\": \"20230509095632|20230509095632\",\r\n  \"EMVTagsIssuer\": \"VISA CREDIT|DIPPED|A0000000031010|8080048000|6800|40|420300|02948BC4A068D32C|2\",\r\n  \"TransactionType\": \"1\",\r\n  \"MerchantInfo\": \"6.04|TOMS & JERRY|زموند|Mall OF DHAHRAN|مركز دهران\",\r\n  \"ResponseCodeDescription\": \"APPROVED\",\r\n  \"ECRReferenceNumber\": \"0987654321654321\",\r\n  \"DF 01\": \"00 02 45 79 3E 01 00 02 45 79 3E 01 00 02 45 7A 3D 01 00 02 45 79 3E 01 00 02 45 79 3E 01 01 74 FF 01 00\",\r\n  \"DF 02\": \"10 34 30 31 36 31 33 2A 2A 2A 2A 2A 37 33 32 39\",\r\n  \"DF 04\": \"06 00 00 00 00 12 00\",\r\n  \"DF 0B\": \"06 30 30 30 33 38\",\r\n  \"DF 0C\": \"0C 32 33 30 35 30 39 30 39 35 36 33 32\",\r\n  \"DF 0E\": \"02 25 10\",\r\n  \"DF 25\": \"0C 33 31 32 39 30 39 35 36 33 32 30 36\",\r\n  \"DF 26\": \"06 33 32 39 32 30 30\",\r\n  \"DF 27\": \"03 30 30 30\",\r\n  \"DF 29\": \"10 39 30 31 31 31 30 30 33 31 32 33 34 35 36 37 38\",\r\n  \"DF 82\": \"02 56 43\",\r\n  \"DF 2A\": \"0F 36 35 30 39 39 39 30 30 30 39 31 31 20 20 20\",\r\n  \"DF 36\": \"06 00 00 00 00 00 00 00\",\r\n  \"DF 87\": \"04 53 41 42 42\",\r\n  \"DF 88\": \"04 35 36 35 31\",\r\n  \"DF 89\": \"1D 32 30 32 33 30 35 30 39 30 39 35 36 33 32\",\r\n  \"DF 90\": \"4E 56 49 53 41 20 43 52 45 44 49 54 7C 44 49 50 50 45 44 7C 41 30 30 30 30 30 30 33 31 30 31 30 7C 38 30 38 30 30 34 38 30 30 30 7C 36 38 30 30 7C 34 30 7C 34 32 30 33 30 30 7C 30 32 39 34 38 42 43 34 41 30 36 38 44 33 32 43 7C 32\",\r\n  \"DF 30\": \"01 31\",\r\n  \"DF 91\": \"3C 36 2E 30 34 7C 54 4F 4D 53 20 26 20 4A 45 52 52 59 7C CF A8 E0 8F 7C 4D 61 6C 6C 20 4F 46 20 44 48 41 48 52 41 4E 7C DC E0 A8 20 DE C4 CE AB D5 A7 C4 7C 7C DE C4 CE AB D5 A7 C4 21\",\r\n  \"DF 28\": \"0F AC A7 E0 8E A5 A8 7C 41 50 50 52 4F 56 45 44\",\r\n  \"DF 81\": \"10 30 39 38 37 36 35 34 33 32 31 36 35 34 33 32 31 87\",\r\n  \"TransactionResponseArabic\": \"مقبول\",\r\n  \"TransactionTypeAsReadable\": \"PURCHASE\",\r\n  \"TerminalVersion\": \"6.04\",\r\n  \"MerchantNameEnglish\": \"TOMS & JERRY\",\r\n  \"MerchantNameArabic\": \"زموند\",\r\n  \"MerchantAddress1English\": \"Mall OF DHAHRAN\",\r\n  \"MerchantAddress1Arabic\": \"مركز دهران\",\r\n  \"MerchantAddress2English\": \"\",\r\n  \"MerchantAddress2Arabic\": \"\",\r\n  \"TransactionRequestDateTime\": \"20230509095632\",\r\n  \"TransactionResponseDateTime\": \"20230509095632\",\r\n  \"CardScheme\": \"VISA CREDIT\",\r\n  \"POSEEntryMode\": \"DIPPED\",\r\n  \"AID\": \"A0000000031010\",\r\n  \"TVR\": \"8080048000\",\r\n  \"TSI\": \"6800\",\r\n  \"CryptResult\": \"40\",\r\n  \"CVMResult\": \"420300\",\r\n  \"ApplicationCryptogram\": \"02948BC4A068D32C\",\r\n  \"CardHolderVerificationMethod\": \"2\",\r\n  \"CardHolderVerificationMethodAsReadable\": \"Card Holder Verified by PIN.\",\r\n  \"CardCode\": \"VC-VISA\",\r\n  \"CardNameEnglish\": \"VISA\",\r\n  \"CardNameArabic\": \"فيزا\",\r\n  \"TransactionResultReasonArabic\": \"مقبول\",\r\n  \"TransactionResultReasonEnglish\": \"APPROVED\"}
  "Event": "OnDataReceive"
}
```

Note:

Scan and Pay Transactions are added in separate section. Please refer Section 9.

7. Installation and Troubleshooting guide

Windows Build (Installation Guidelines)

1. Download and extract Geidea Files and save it in any location in your computer.

 Geidea Files	21-01-2026 11:14	Compressed (zipp...	67,434 KB
--	------------------	---------------------	-----------

After extracting the ECR ZIP file into the chosen directory, the following files will be available.

 Geidea Web ECR-27102025	18-01-2026 12:27	File folder	
 INSTALL-Service	19-01-2026 11:42	Windows Batch File	6 KB
 UNINSTALL-Service	19-01-2026 11:57	Windows Batch File	4 KB

2. You will find one folder and two files as listed below:
 - a. **Geidea Web ECR-DDMMYYYY** – Folder
 - b. **INSTALL-Service** – Batch file used to install the service
 - c. **UNINSTALL-Service** – Batch file used to uninstall the service
3. Double-click the **INSTALL-Service** batch file. It will prompt for Administrator permission. After providing the credentials, you will be required to enter the following details:
 - 3.1 **Service Name** – Description of the service (**Ex: GeideaECRService-KSA**)
Use GeideaECRService-KSA as the service name for consistency.
 - 3.2 **Display Name** – Name of the service as it will appear in the Windows Services list (**Example: Geidea ECR Core Service KSA**)
Use Geidea ECR Core Service KSA as the display name for Consistency.
 - 3.3 **Path** – Location of the **Geidea Web ECR-DDMMYYYY** folder (open the folder and copy its full path)
Example:
E:\WEB ECR Production Files\KSA\Geidea Files\Geidea Web ECR-DDMMYYYY

```

Administrator: Geidea ECR Service Installer

=====
GEIDEA ECR SERVICE INSTALLER
=====

[*] Running with administrator privileges

=====
SERVICE CONFIGURATION
=====

Enter the SERVICE NAME (internal name, no spaces recommended):
Example: GeideaECRService, GeideaPaymentAPI

Service Name: GeideaECRService-KSA

Enter the DISPLAY NAME (what users see in services.msc):
Example: Geidea ECR Core Web Integration

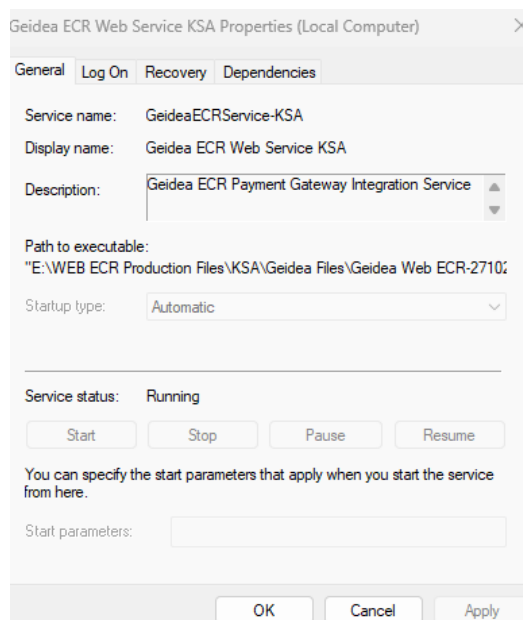
Display Name: Geidea Web ECR Service KSA

Enter the SERVICE PATH (full path to the folder):
Example: E:\WEB ECR Production Files\KSA\Geidea Web ECR-27102025

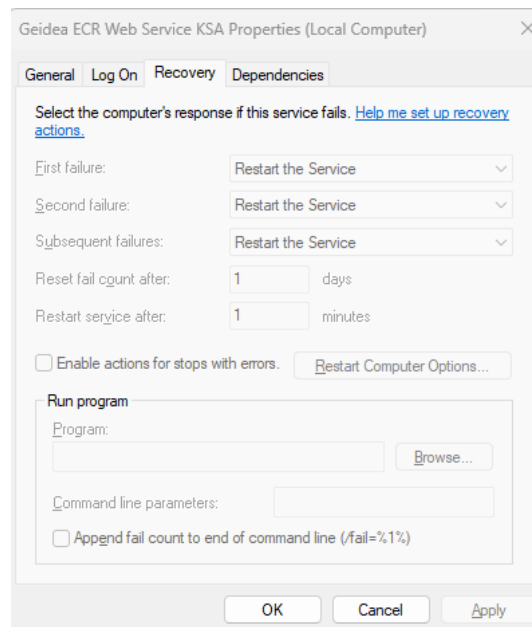
Service Path: E:\WEB ECR Production Files\KSA\Geidea Files\Geidea Web ECR-27102025

```

4. The service will be installed with the startup type set to **Automatic**, meaning it will start automatically when the system boots.
5. Verify that the startup type is set to **Automatic** or **Automatic (Delayed Start)** by checking the service **Properties** → **General** tab



6. Verify that the recovery is set to **Restart the Service** for first, second and subsequent failures.



7. After applying the above steps you can open the browser and enter localhost:5000 to use Geidea Web ECR Testing Tool.

8. SAMA and Terminal Response Messages

8.1 Terminal Response Messages

There are 3 types of messages you can receive before the transaction is sent from POS to Saudi Payment.

- ECR Communication messages – This is not an Error code.
- Error Codes 1 – Occurs before the present card screen.
- Error Codes 2 – Occurs after the present card screen

1. ECR Communication messages

Si.No	Message	Terminal Action Message
1	Swipe or Insert	"SWIPE_OR_INSERT"
2	Card Inserted	"CARD_INSERTED"
3	Card Removed	"CARD_REMOVED"
4	Pin Entry	"PIN_ENTRY"
5	Connecting	"CONNECTING"
6	Re-Connect	"RE-CONNECT"
7	Sending Request	"SENDING_REQUEST"
8	Waiting Response	"WAITING_RESPONSE"
9	SAF Processing	"SAF_PROCESSING"
10	Connection Made	"CONNECTION_MADE"
11	Reconciliation Successful	"RECONCILLATION_SUCCESFUL"
12	Connection Closed	"CONNECTION_CLOSED"

2. Error Codes 1– Before “Present Card Screen”.

Si.No	Message	Terminal Action Message
1	Swipe or Insert	"SWIPE_OR_INSERT"
2	Card Inserted	"CARD_INSERTED"
3	Card Removed	"CARD_REMOVED"
4	Pin Entry	"PIN_ENTRY"
5	Connecting	"CONNECTING"
6	Re-Connect	"RE-CONNECT"
7	Sending Request	"SENDING_REQUEST"
8	Waiting Response	"WAITING_RESPONSE"
9	SAF Processing	"SAF_PROCESSING"
10	Connection Made	"CONNECTION_MADE"
11	Reconciliation Successful	"RECONCILLATION_SUCCESFUL"
12	Connection Closed	"CONNECTION_CLOSED"

3. Error Codes 2- After “Present Card Screen”.

Si.No	Message	Terminal Action Message
1	"User Cancelled and Timeout",	USER_CANCELLED_AND_TIMEOUT
2	"Card Read Error",	CARD_READ_ERROR
3	"Data Error",	DATA_ERROR
4	"Blocked Card",	BLOCKED_CARD
5	"No Active Application Found",	NO_ACTIVE_APPLICATION_FOUND
6	"Insert Card Only Use Smart Card Reader",	INSERT_CARD_ONLY_USE_SMART_CARD_READER
7	"Maximum Amount Limit Exceeded",	MAXIMUM_AMOUNT_LIMIT_EXCEEDED
8	"Pin Quit",	PIN_QUIT
9	"Transaction Cancelled",	TRANSACTION_CANCELLED
10	"Wrong Pin",	WRONG_PIN
11	"Retry Limit Exceeded",	RETRY_LIMIT_EXCEEDED
12	"Maximum SAF Reached",	MAXIMUM_SAF_REACHED
13	"Mag Stripe Not Supported",	MAG_STRIPE_NOT_SUPPORTED
14	"Card Scheme Not Supported",	CARD_SCHEME_NOT_SUPPORTED
15	"DE-SAF Failed",	DE_SAF_FAILED
16	"Reconciliation No Business",	RECONCILIATION_NO_BUSINESS
17	"Reconciliation Failed",	RECONCILIATION_FAILED
18	"No Txn to Reverse",	NO_TXN_TO_REVERSE

19	"Try Contacting Interface",	TRY_CONTACTING_INTERFACE
20	"Supervisor Password Quit",	SUPERVISOR_PASSWORD_QUIT
21	"Gprs Down",	GPRS_DOWN
22	"Refund PAN Not Matching",	REFUND_PAN_NOT_MATCHING
23	"Invalid Pawword",	INVALID_PAWWORD
24	"Card Not Detected",	CARD_NOT_DETECTED

Note:

"Idle Screen" terminal action will be received for all the cases which indicate the completion of a transaction.

TerminalAction:" IDLE_SCREEN"

Terminal Busy

If the Terminal Busy you will receive BUSY message under the event terminal status.

"EventName":"TERMINAL_STATUS",

"TerminalStatus":"BUSY"

8.2 SAMA Response Messages

After completion of the transaction terminal will send SAMA response codes this can be obtained from:

```
"EventName": "TERMINAL_RESPONSE",
  "JsonResult": "...TransactionResponse:***..."

*** - indicates sama Response code.
```

Based on the response codes we have only 2 cases “APPROVED” or “DECLINED” that you can get from the below table.

SAMA Response Codes:

Si.No	Code	Description	Action
1	000	Approved	Approved
2	001	Honour with identification	Approved
3	003	Approved(VIP)	Approved
4	007	Approved, update ICC (To be used when a response includes an issuer script. This code is used for SPAN only.)	Approved
5	087	Offline Approved(Chip Only)	Approved
6	089	Unable to go On-line approved (Chip only)	Approved
7	400	Accepted(For Reversal Case)	Approved
8	500	Reconciled, Total Matched	Approved
9	501	Reconciled, Total UnMatched	Approved
10	***	For other 3 digits	Declined